Mastering Arrays

Your comprehensive guide to one of programming's most essential data structures — from fundamentals to advanced techniques.

Made with GAMMA

What is an Array?

An array is a **fundamental data structure** that stores an ordered collection of elements. Think of it as a row of labeled boxes — each slot holds a value and can be accessed by its position.

Ordered

Elements maintain their sequence

Indexed

Each element has a numeric position

Flexible

Can hold any data type

Made with GAMMA

Declaring & Initializing Arrays

Before using an array, you must **declare** it — then **initialize** it with values. JavaScript offers two clean approaches.

Literal Notation

The most common and concise approach.

```
let daysOfWeek = [
  "Monday",
  "Tuesday",
  "Wednesday"
];
```

Constructor Syntax

Uses the Array constructor explicitly.

```
let numbers = new Array(1, 2, 3);

let empty = new Array();
let sized = new Array(5);
```

Made with GAMMA

The Power of Indexing

Array elements are accessed using **zero-based indices** — the first element is at position `0`, the second at `1`, and so on.

Access by Index

`daysOfWeek[0]` → `"Monday"`

Modify by Index

`daysOfWeek[1] = "Fri";`

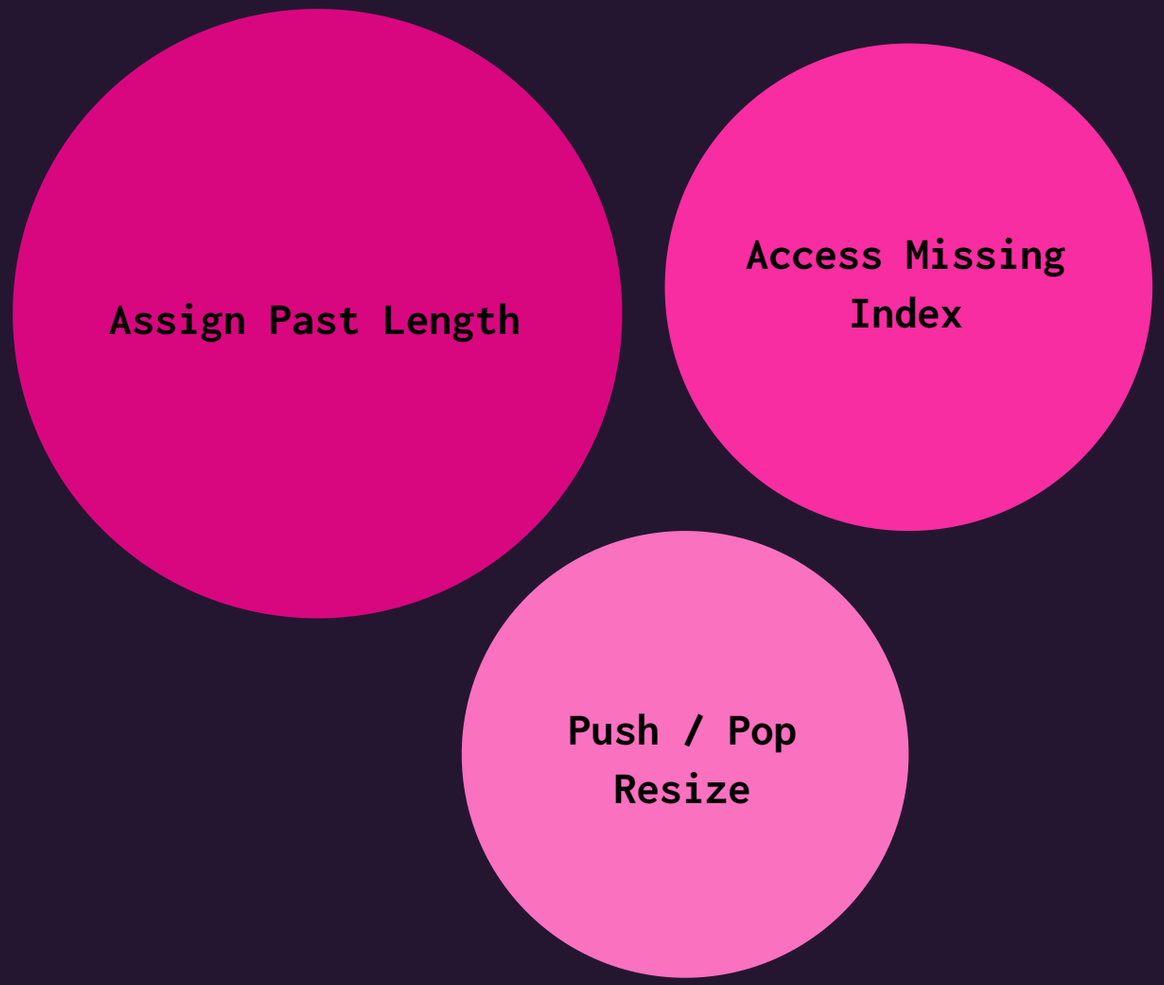
Last Element

`daysOfWeek[daysOfWeek.length - 1]`

Made with GAMMA

Dynamic Arrays: Flexibility in Action

Unlike arrays in some languages, **JavaScript arrays are dynamic** — they grow and shrink at runtime without pre-allocating fixed memory.



This flexibility makes JavaScript arrays ideal for situations where the data size isn't known in advance.

Made with GAMMA

Manipulating Arrays

Arrays come with a rich set of **built-in methods** that make adding, removing, and transforming data effortless.

Add Elements

`push()` adds to the end; `unshift()` adds to the start.

Remove Elements

`pop()` removes from the end; `shift()` removes from the start.

Splice & Replace

`splice()` can insert, remove, or replace elements at any index.

Search

Linear search scans all; binary search (sorted arrays) cuts the search space in half each step.

Made with GAMMA

Beyond the Basics

Multidimensional Arrays

Arrays of arrays — perfect for grids, matrices, and game boards.

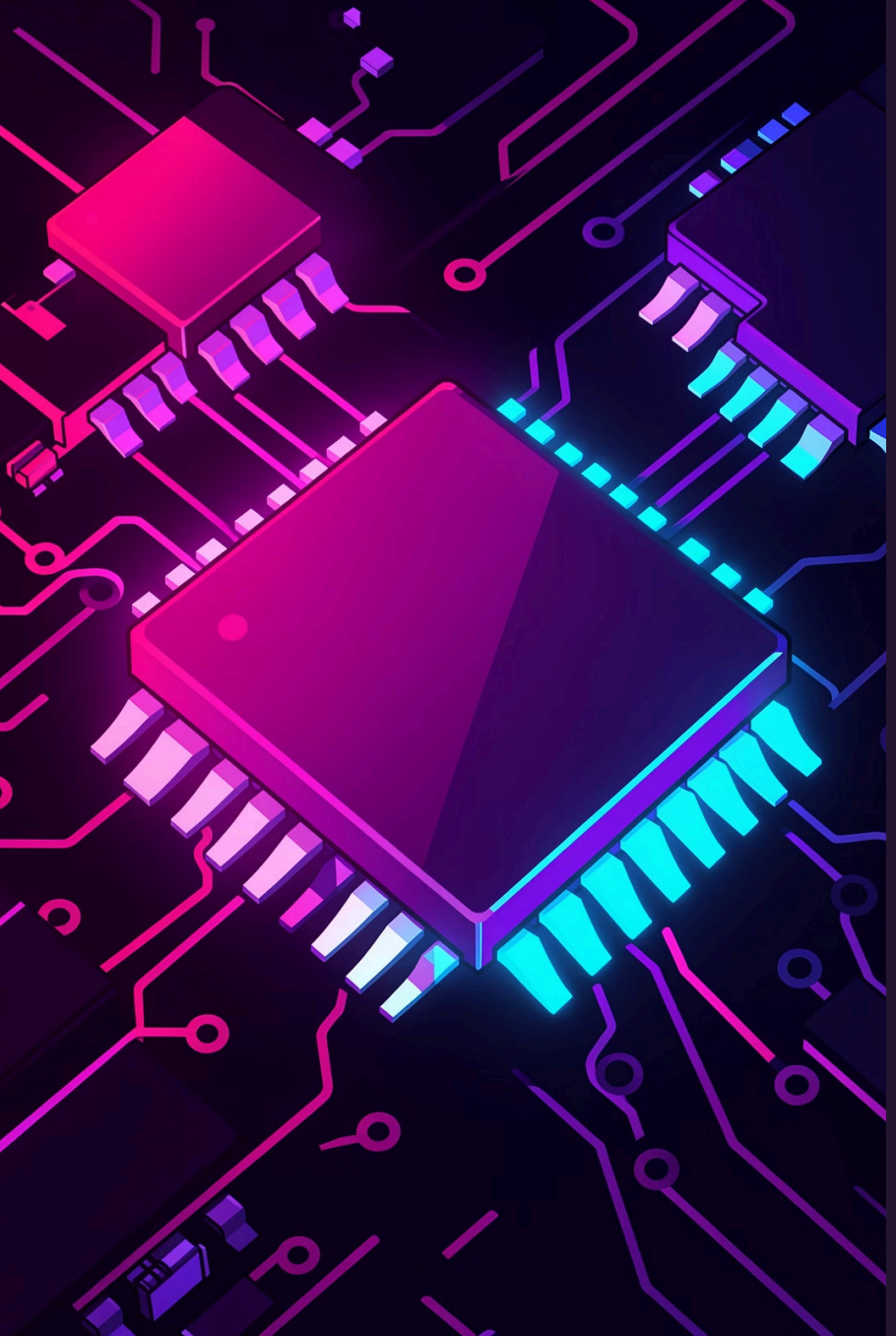
```
// Tic-Tac-Toe board
let board = [
  ['X', 'O', 'X'],
  [' ', 'O', ' '],
  [' ', ' ', 'X']
];
```

Parallel Arrays

Multiple arrays where elements at the same index are related — an alternative to objects when simplicity matters.

```
let names = ["Alice", "Bob"];
let ages = [25, 30];
// names[0]
```

Made with GAMMA



Memory Occupation

JavaScript arrays are **sparse by design** — only assigned elements consume meaningful memory. Declaring a large array does not immediately allocate all that space.

Declared but Empty

`let arr = new Array(1000);` — minimal memory used until values are assigned.

Populated Slots

Only indices you explicitly assign occupy real memory space.

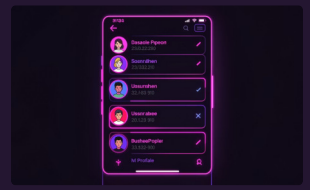
Sparse Arrays

Skipping indices creates "holes" — they exist but hold no data.

Made with GAMMA

Practical Applications

Arrays are everywhere in real-world programming. Here's where they shine.



Data Lists

Store users, products, scores, or any collection of items.

Stacks & Queues

Implement classic data structures using `push/pop` and `shift/unshift`.

Data Processing

Read, transform, and filter data from files or user inputs.

Dynamic UIs

Render lists, grids, and interactive components from array data.

Made with GAMMA

Arrays – Your Essential Programming Tool

Arrays are **versatile, efficient, and indispensable**. Mastering them is the foundation for writing cleaner, more powerful code.

Store

Organize collections of data

Access

Retrieve values by index instantly

Manipulate

Transform data with built-in methods

Build

Create real-world applications

Made with GAMMA